

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Question 1

```
SELECT S.StudentName,  
       C.ClassName,  
       T.TeacherName,  
       G.Grade  
FROM Student S  
INNER JOIN Class C  
ON S.ClassID = C.ClassID  
INNER JOIN Teacher T  
ON C.TeacherID = T.TeacherID  
INNER JOIN Grade G  
ON S.StudentID = G.StudentID;
```

studentname	classname	teachername	grade
ARUN	III Year IT	Mr.Kumar	A
RAHUL	III Year IT	Mr.Kumar	A+
PRIYA	II Year CSE	MS.Meena	B+
SNEHA	I Year ECE	Mrs.Lakshmi	B

Question 2

```
SELECT D.DoctorID,  
       D.DoctorName,  
       COUNT(A.AppointmentID) AS TotalAppointments  
FROM Doctor D  
INNER JOIN Appointment A  
ON D.DoctorID = A.DoctorID  
WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())  
AND YEAR(A.AppointmentDate) = YEAR(CURDATE())  
GROUP BY D.DoctorID, D.DoctorName  
HAVING COUNT(A.AppointmentID) > 10;
```

DoctorID	DoctorName	TotalAppointments
D1	Dr. Kumar	11

Question 3

```
SELECT ProductID,  
       ProductName  
FROM Product  
WHERE ProductID NOT IN  
(  
    SELECT ProductID  
    FROM Orders  
);
```

ProductID	ProductName
P4	Monitor
P5	Printer

Question 4

```
SELECT DISTINCT S.StudentName
FROM Student S
JOIN Enrollment E1
ON S.StudentID = E1.StudentID
JOIN Course C1
ON E1.CourseID = C1.CourseID
JOIN Enrollment E2
ON S.StudentID = E2.StudentID
JOIN Course C2
ON E2.CourseID = C2.CourseID
WHERE C1.CourseName = 'DBMS'
AND C2.CourseName = 'OS';
```

StudentName
ARUN
RAHUL

Question 5

```
CREATE VIEW BelowDeptAverage AS
SELECT E.EmpID,
       E.EmpName,
       E.DeptID,
       E.Salary
FROM Employee E
WHERE Salary <
(
    SELECT AVG(Salary)
    FROM Employee
    WHERE DeptID = E.DeptID
);
```

```
mysql> SELECT * FROM BelowDeptAverage;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | D1     | 45000.00 |
| 103   | Priya   | D2     | 40000.00 |
| 105   | Vijay   | D1     | 50000.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
Query OK, 3 rows affected, 1 warning (0.01 sec)  
Rows matched: 3  Changed: 3  Warnings: 1
```

```
mysql> SELECT * FROM Employee;
```

EmpID	EmpName	DeptID	Salary
101	Arun	D1	49500.00
102	Rahul	D1	60000.00
103	Priya	D2	44000.00
104	Sneha	D2	52000.00
105	Vijay	D1	55000.00

```
5 rows in set (0.00 sec)
```

```
mysql> |
```

Question 6

```
SELECT C.CustomerName,  
       A.AccountNo,  
       B.BranchName,  
       BT.TransactionType,  
       BT.Amount  
FROM Customer C  
INNER JOIN Account A  
ON C.CustomerID = A.CustomerID  
INNER JOIN Branch B  
ON A.BranchID = B.BranchID  
INNER JOIN BankTransaction BT  
ON A.AccountNo = BT.AccountNo;
```

CustomerName	AccountNo	BranchName	TransactionType	Amount
Arun	A101	Chennai	Deposit	10000.00
Rahul	A103	Chennai	Deposit	20000.00
Priya	A102	Coimbatore	Withdrawal	5000.00

3 rows in set (0.00 sec)

Question 7

```
SELECT Category,  
       SUM(Amount) AS TotalSales  
FROM Sales  
GROUP BY Category  
HAVING SUM(Amount) > 10000;
```

Category	TotalSales
Electronics	18000.00
Furniture	17000.00

2 rows in set (0.00 sec)

Question 8

```
SELECT DISTINCT M.MemberName  
FROM Member M  
WHERE NOT EXISTS  
(  
    SELECT *  
    FROM  
    (  
        SELECT DISTINCT Genre  
        FROM Book  
    ) G  
    WHERE NOT EXISTS  
    (  
        SELECT *  
        FROM Borrow BR  
        JOIN Book B  
        ON BR.BookID = B.BookID  
        WHERE BR.MemberID = M.MemberID  
        AND B.Genre = G.Genre  
    )  
);
```

```
mysql> SELECT DISTINCT M.MemberName  
-> FROM Member M  
-> WHERE NOT EXISTS (...);
```

MemberName
Arun

1 row in set (0.00 sec)

Question 9**SELECT * FROM Inventory;**

```
mysql> SELECT * FROM Inventory;
```

ProductID	ProductName	Quantity	Price	SupplierID
P101	Laptop	10	55000.00	S1
P102	Mouse	50	500.00	S1
P103	Keyboard	30	1200.00	S2
P104	Monitor	15	12000.00	S2

Question 10

Normalize the given Student-Course Enrollment data up to Third Normal Form (3NF).

UNF (Unnormalized Form)

StudentID	StudentName	CourseID	CourseName	Faculty
101	Arun	C101	DBMS	Dr. Kumar
101	Arun	C102	Operating Systems	Dr. Meena
102	Priya	C101	DBMS	Dr. Kumar
103	Rahul	C103	Python	Dr. John

First Normal Form (1NF)

Separate the repeating groups so that each row contains only one course.

Student Table

StudentID	StudentName
101	Arun
102	Priya
103	Rahul

Enrollment Table

StudentID	CourseID
101	C101
101	C102
102	C101
103	C103

Second Normal Form (2NF)

Move the course details into a separate table to remove partial dependency.

Course Table

<i>CourseID</i>	<i>CourseName</i>	<i>Faculty</i>
<i>C101</i>	<i>DBMS</i>	<i>Dr. Kumar</i>
<i>C102</i>	<i>Operating Systems</i>	<i>Dr. Meena</i>
<i>C103</i>	<i>Python</i>	<i>Dr. John</i>

Enrollment Table

<i>StudentID</i>	<i>CourseID</i>
<i>101</i>	<i>C101</i>
<i>101</i>	<i>C102</i>
<i>102</i>	<i>C101</i>
<i>103</i>	<i>C103</i>

Third Normal Form (3NF)

The final normalized database consists of three tables:

Student

<i>StudentID</i>	<i>StudentName</i>
<i>101</i>	<i>Arun</i>
<i>102</i>	<i>Priya</i>
<i>103</i>	<i>Rahul</i>

Course

<i>CourseID</i>	<i>CourseName</i>	<i>Faculty</i>
<i>C101</i>	<i>DBMS</i>	<i>Dr. Kumar</i>
<i>C102</i>	<i>Operating Systems</i>	<i>Dr. Meena</i>
<i>C103</i>	<i>Python</i>	<i>Dr. John</i>

Enrollment

<i>StudentID</i>	<i>CourseID</i>
<i>101</i>	<i>C101</i>
<i>101</i>	<i>C102</i>
<i>102</i>	<i>C101</i>
<i>103</i>	<i>C103</i>

Code of Conduct:

I K. VIJAYALAKSHMI (1 9 2 5 1 1 1 8 9) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.VIJAYALAKSHMI

